



Hello! How is it going?



so tired sleepy
good

hello fine zehr gut

not good cause of inet

wonderful

Agenda

- Recall last session
- Discuss current topic
- Practice
- Quiz

Total Recall



What do you remember most from few previous workshops? (short answer)

timezone

duration

LocalDate

ZonedDateTime, Instant, and
other magic with dates

What does ZonedDateTime represent?

0 ✗

A formatted time zone

3 ✓

Universal time format

7 ✗

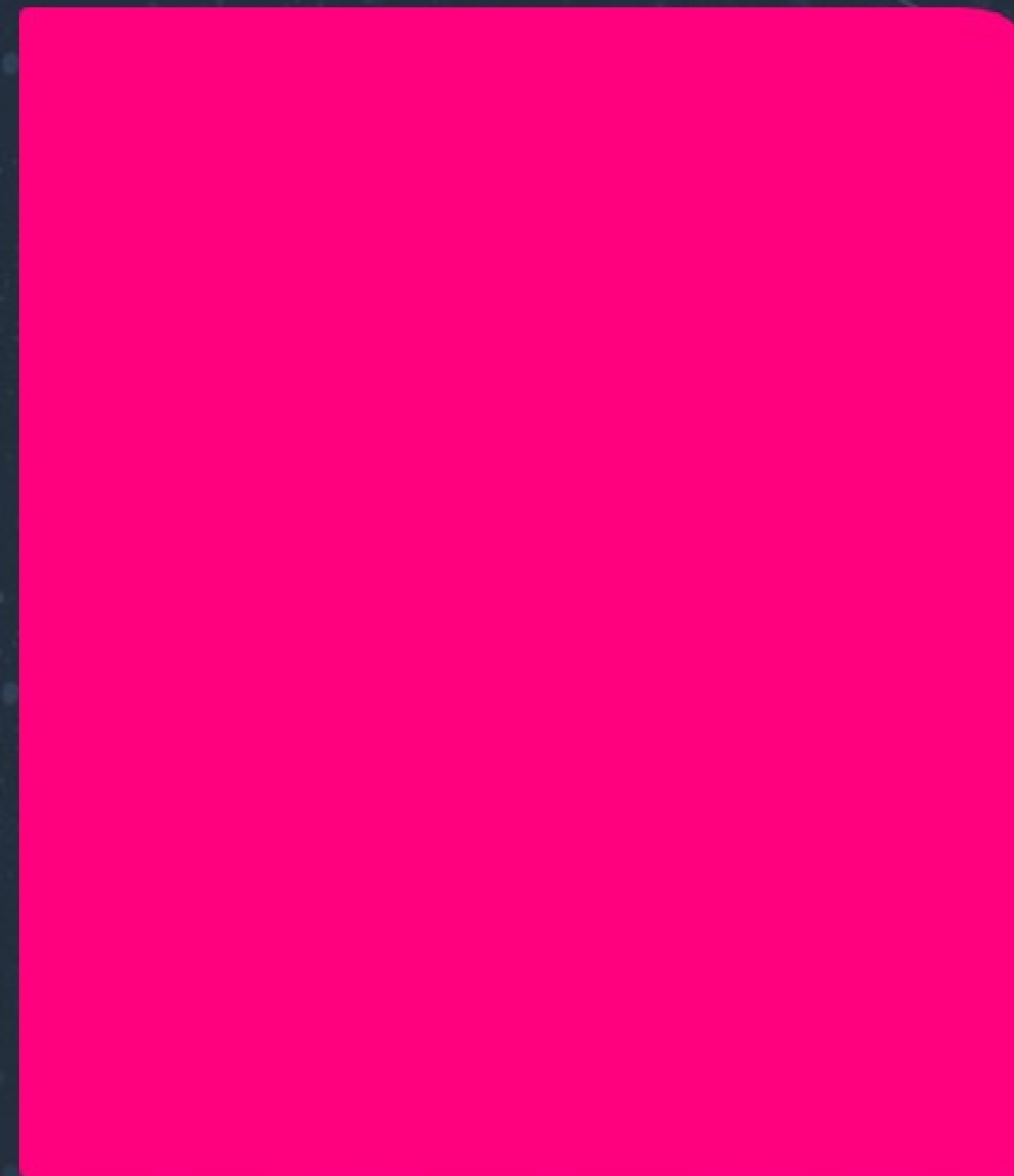
**Date and time in a specific
timezone

0 ✗

Formatted string output

What does `ZoneId.systemDefault()` return?

9 ✓



0 ✗

A formatted date

0 ✗

A timezone-aware
timeline

0 ✗

Specific date intervals

Which package does LocalDate belong to in Java?

1 ✓



0 ✗

java.parse.date

0 ✗

java.timezone

0 ✗

java.local.time

How can you add 5 days to LocalDate?

2 ✓



Use the `plusDays(5)` method

0 ✗

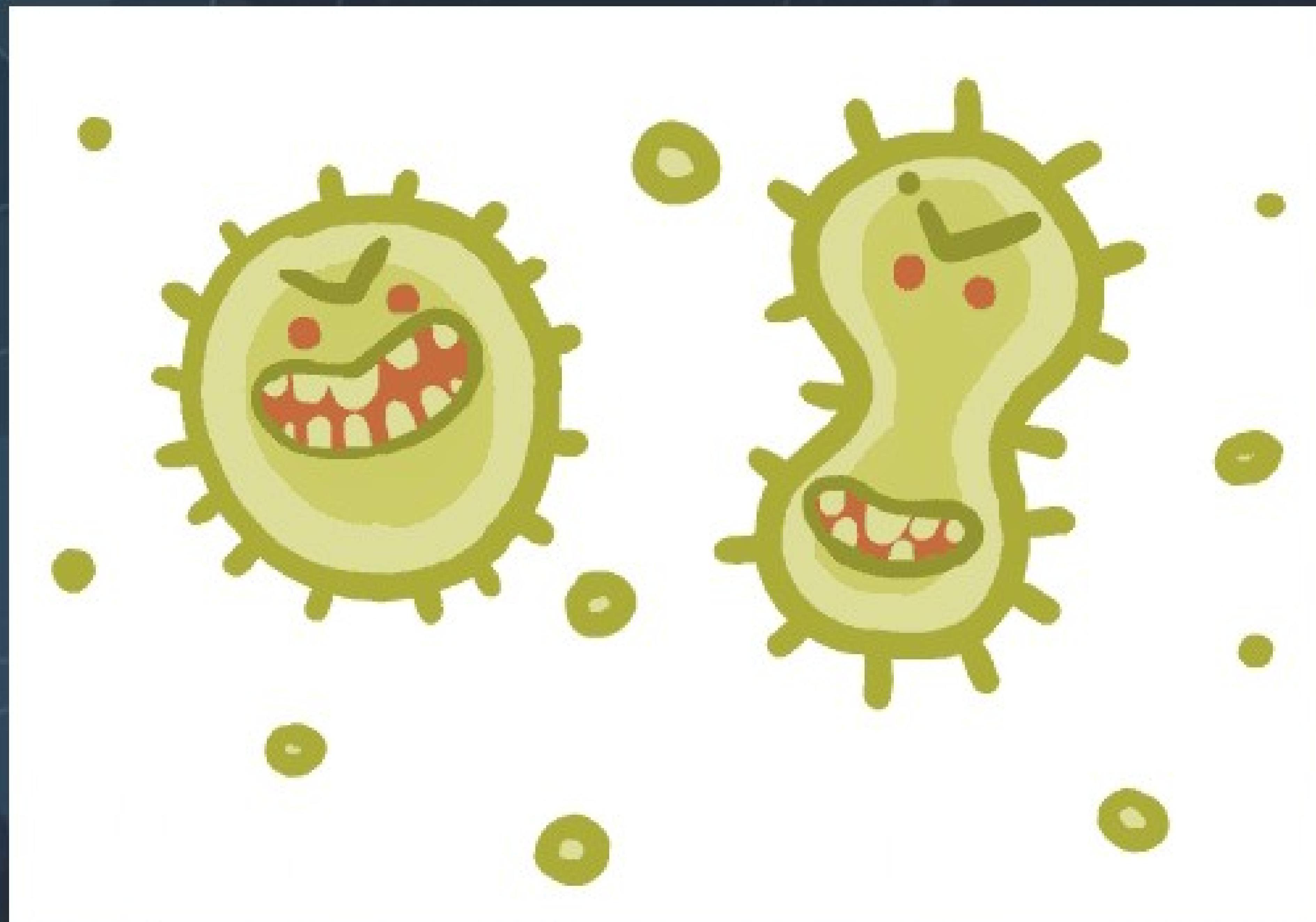
Call print function

0 ✗

Direct modify date instance

0 ✗

Handle zone transitions



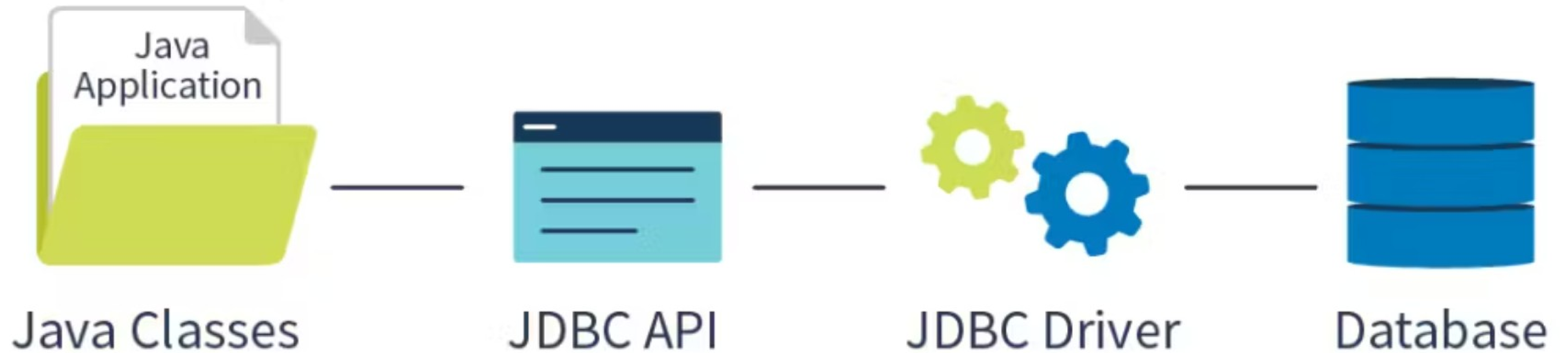
Next topic

Random fact

All the world's bacteria stacked on top of each other would stretch for 10 billion light-years. Together, Earth's 0.001mm-long microbes could wrap around the Milky Way over 20,000 times.

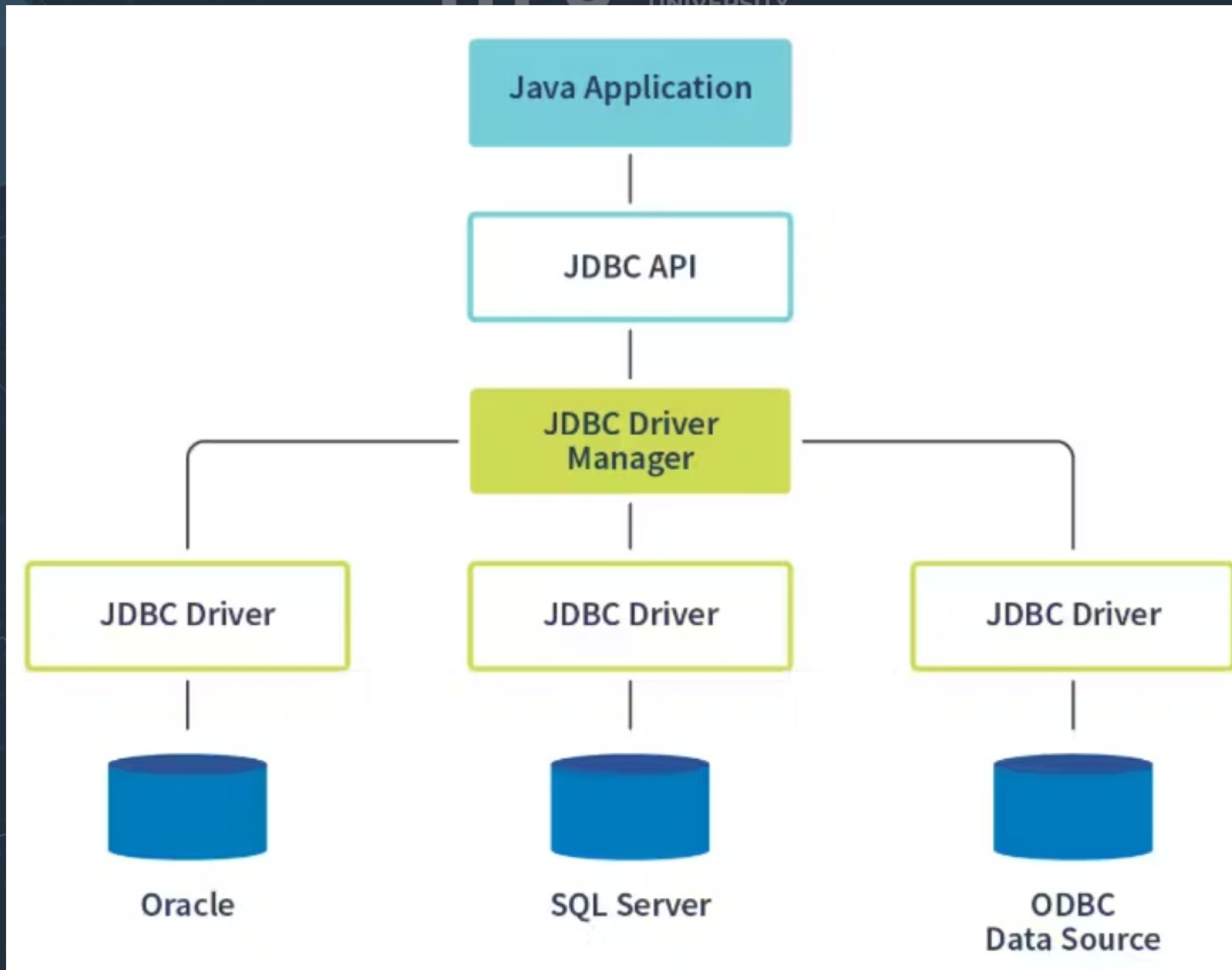
#1 Introduction to JDBC

1.



A database session can be created using the connection driver class directly, but the preferred implementation is to use the DriverManager class, which registers all applicable drivers.

*** In the DriverManager class, you cannot register two different driver versions to the same database.



Common JDBC Components

- **java.sql.Connection:**

The Connection interface represents a connection to a particular database. It is responsible for establishing and maintaining a connection to the database server. The Connection object provides methods for executing SQL statements, managing transactions, retrieving metadata, and managing other aspects of a database connection.

- **java.sql.Statement**

The Statement interface is used to execute simple SQL statements. It can perform queries and updates but does not support parameterization.

- **java.sql.PreparedStatement**

The PreparedStatement interface extends SQL statements and allows you to execute parameterized prepared statements.

Common JDBC Components

- **java.sql.CallableStatement**

The CallableStatement interface is used to execute stored procedures or database functions. It provides methods for setting input and output parameters and calling a stored procedure.

- **java.sql.ResultSet**

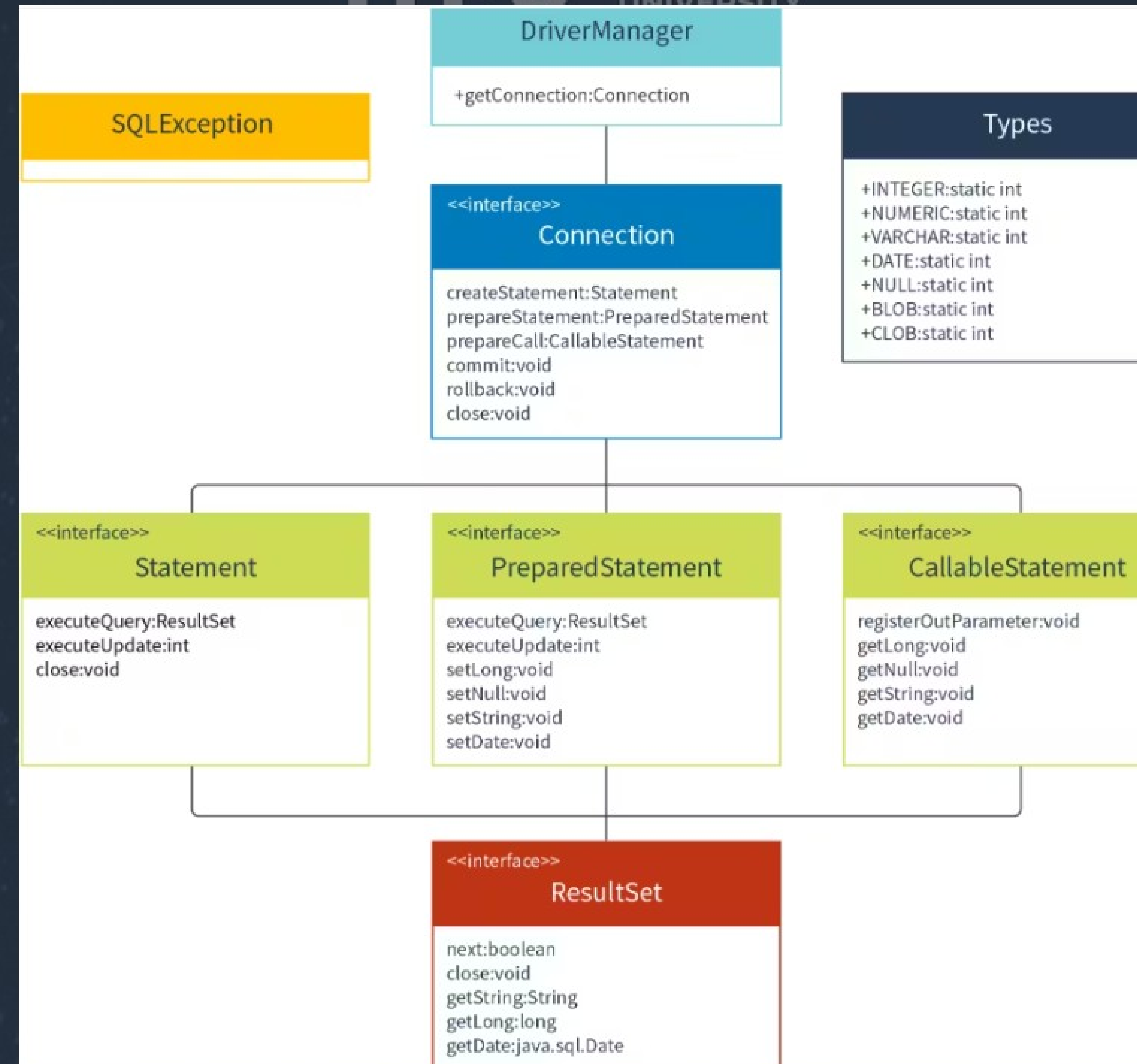
The ResultSet interface represents the result of a database query. It provides methods for navigating a result set, accessing column values, and performing operations such as iterating over rows, getting data by index or column name, and updating data.

- **java.sql.SQLException**

The SQLException class is responsible for managing errors that arise in a database application.

- **java.sql.Types**

The Types class defines a set of constants that map to SQL data types defined in database systems.



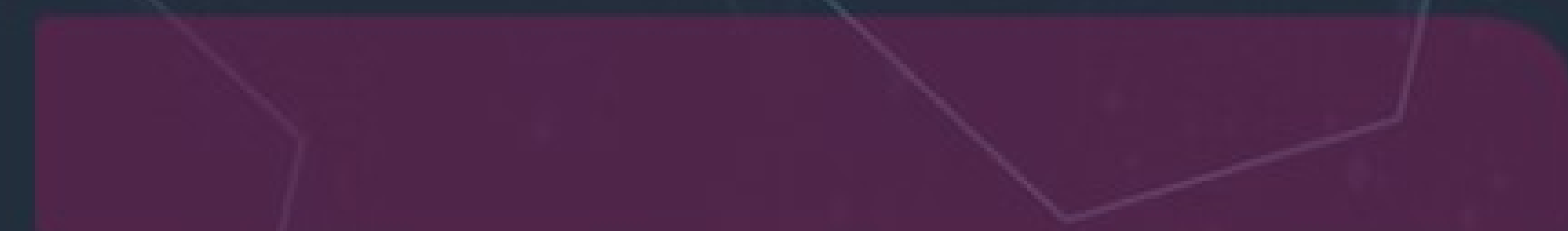
A diagram of the main JDBC classes and interfaces is shown below.

What does JDBC stand for?

9 ✓



1 ✗



Java Database Control

0 ✗



Java Database Class

1 ✗



Java Database Connector

#1.2 Creating a JDBC Application

@ introExample package

two options of the same thin

Year	JDBC Version	Java Specification Requests (JSR)	JDK Implementation
2017	JDBC 4.3	JSR 221	Java SE 9
2014	JDBC 4.2	JSR 221	Java SE 8
2011	JDBC 4.1	JSR 221	Java SE 7
2006	JDBC 4.0	JSR 221	Java SE 6
2001	JDBC 3.0	JSR 54	JDK 1.4
2001	JDBC 3.0		JDK 1.2
2001	JDBC 1.2		JDK 1.1

Types of Drivers:

Type 1: JDBC-ODBC Bridge

The first type of driver, the JDBC-ODBC bridge, uses the ODBC (Open Database Connectivity) driver provided by the database vendor. It acts as a bridge between the JDBC API and the ODBC API. This type of driver allows Java applications to access databases that only provide an ODBC connection but do not have direct JDBC support. The advantage of the type 1 driver is that it can provide basic database connectivity for Java applications, regardless of the database system. However, it relies on ODBC, which can introduce additional dependencies and potential performance losses. It is typically used when no other JDBC driver exists for a particular database.

Types of Drivers:

Type 2: Native API Partly Java Driver

The type 2 driver combines the use of native database client libraries with a Java-based driver component. The driver translates JDBC API calls into native API calls to the database system for which it is intended. This type of driver can provide better performance than type 1 because it interacts directly with native database client libraries. However, this requires database-specific client libraries to be installed on the client machine, making it less portable. A type 2 is useful when direct access to a native API is required for performance reasons or when a pure Java driver is unavailable for a particular database.

Types of Drivers:

Type 3: Network-Protocol Pure Java Driver

The type 3 driver communicates with the middleware server using a network protocol. The middleware server, in turn, connects to the database using the appropriate driver for that specific database. The type 3 driver acts as an intermediate layer between the Java application and the database server. It may provide better portability because it is implemented entirely in Java and does not require a database client to be installed on the client machine. The type 3 driver is useful when an application needs to access multiple databases or when direct access to the database server is impossible due to security or network configuration restrictions.

Types of Drivers:

Type 4: Native-Protocol Pure Java Driver

A type 4 driver communicates directly with the database server using a database-specific network protocol. It is fully implemented in Java and does not require any additional software or client installation. This type of driver provides the best performance and portability because it eliminates the need for native client libraries and works across platforms. This is the most common type of driver in modern JDBC applications. A type 4 driver is recommended when an application requires optimal performance and maximum portability between different database systems.

#2.1 Executing Database Queries

JDBC Versions Prior to 4.0

In early JDBC implementations, a driver is loaded and registered as follows:

```
java.sql.DriverManager.registerDriver(new com.ibm.as400.access.AS400JDBCDriver());
```

If you need to manually load the driver class into the memory, you can do so as follows:

```
forName("org.gjt.mm.mysql.Driver");  
forName("sun.jdbc.odbc.jdbcodbcDriver");
```


JDBC 4.0 and Later Versions

Beginning with JDBC 4.0 (Java SE 6), the driver registration process became optional for type 4 drivers. Instead of explicitly loading and registering a driver, JDBC introduced a service provider mechanism that allows drivers to be automatically discovered and registered. Drivers that are compatible with JDBC 4.0 or later versions can be detected and registered automatically if they are present on the application's classpath.

```
java -Djdbc.drivers=com.example.jdbc.Driver MyApp
```

or

```
System.setProperty("jdbc.drivers", "com.example.jdbc.Driver");
```


Creating Objects for Sending and Executing Requests

- **Statement**—Simple SQL Queries
- **PreparedStatement**—Prepared (or Parameterized) SQL Queries
- **CallableStatement**—Stored Procedures

Statement:

- **createStatement()**

To create a Statement instance on the Connection object, the createStatement() method is called. The Statement object acts as a container for SQL commands.

- **executeQuery()**

To execute SELECT queries, the executeQuery() method of the Statement object is called, and the result of the selection is provided in the ResultSet object.

- **executeUpdate()**

To execute INSERT, UPDATE, and DELETE queries (and others that change the state of a database), the executeUpdate() method of the Statement object is called.

- **close()**

Once a Statement object is no longer going to be used, it is important to close it by calling the close() method to release resources and prevent them from being leaked.

PreparedStatement:

A prepared (or parameterized) database query is a type of SQL query that allows you to define special parameters (placeholders) for dynamic values in the query. Such parameters are already associated with the actual values during execution.

```
SELECT * FROM my_table WHERE column1 = ? AND column2 = ?
```


CallableStatement

A precompiled collection of one or more SQL statements that are stored in a database is known as a stored procedure. Stored procedures are usually written in a procedural language specific to the database system. They are saved in the database itself and can be executed on demand. Stored procedures enable code reuse, performance improvements, security enhancements, and more.

How do you execute a query using a PreparedStatement object in JDBC?

7 ✓

1 ✗

`preparedStatement.executePreparedQuery()`

`preparedStatement.executeQuery()`

0 ✗

`preparedStatement.runQuery()`

0 ✗

`preparedStatement.runPreparedQuery()`

#2.2 Retrieving Query Results

executeQuery()

```
ResultSet rs = statement.executeQuery("SELECT * FROM Books");
```

```
String isbn = rs.getString(1);  
float price = rs.getDouble("Price");
```

```
while (rs.next()) {  
}
```


SQL	JDBC/Java	setXXX	updateXXX
VARCHAR	java.lang.String	setString	updateString
CHAR	java.lang.String	setString	updateString
LONGVARCHAR	java.lang.String	setString	updateString
BIT	boolean	setBoolean	updateBoolean
NUMERIC	java.math.BigDecimal	setBigDecimal	updateBigDecimal
TINYINT	byte	setByte	updateByte
SMALLINT	short	setShort	updateShort
INTEGER	int	setInt	updateInt
BIGINT	long	setLong	updateLong
REAL	float	setFloat	updateFloat
FLOAT	float	setFloat	updateFloat
DOUBLE	double	setDouble	updateDouble
VARBINARY	byte[]	setBytes	updateBytes
BINARY	byte[]	setBytes	updateBytes
DATE	java.sql.Date	setDate	updateDate
TIME	java.sql.Time	setTime	updateTime
TIMESTAMP	java.sql.Timestamp	setTimestamp	updateTimestamp
CLOB	java.sql.Clob	setClob	updateClob
BLOB	java.sql.Blob	setBlob	updateBlob
ARRAY	java.sql.Array	setARRAY	updateARRAY
REF	java.sql.Ref	SetRef	updateRef
STRUCT	java.sql.Struct	SetStruct	updateStruct

Scrolling Using ResultSet


```
Statement stat = conn.createStatement(TYPE_SCROLL, CONCUR_UPDATABLE);
```

```
PreparedStatement stat = conn.prepareStatement(command, TYPE_SCROLL, CONCUR);
```

```
Statement stat = conn.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,  
ResultSet.CONCUR_READ_ONLY);
```


Type	Description
ResultSet.TYPE_FORWARD_ONLY	The cursor can only move forward in the result set.
ResultSet.TYPE_SCROLL_INSENSITIVE	The cursor can scroll both forward and backward, and the result set is not affected by any changes made to the database by other users after the result set was created.
ResultSet.TYPE_SCROLL_SENSITIVE	The cursor can scroll both forward and backward, and the result set is affected by any changes made to the database by other users after the result set was created.

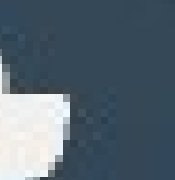
The possible values for the TYPE_SCROLL parameter

Concurrency	Description
ResultSet.CONCUR_READ_ONLY	The parameter value creates a read-only result set. This is the default.
ResultSet.CONCUR_UPDATABLE	The parameter value creates an updateable result set.

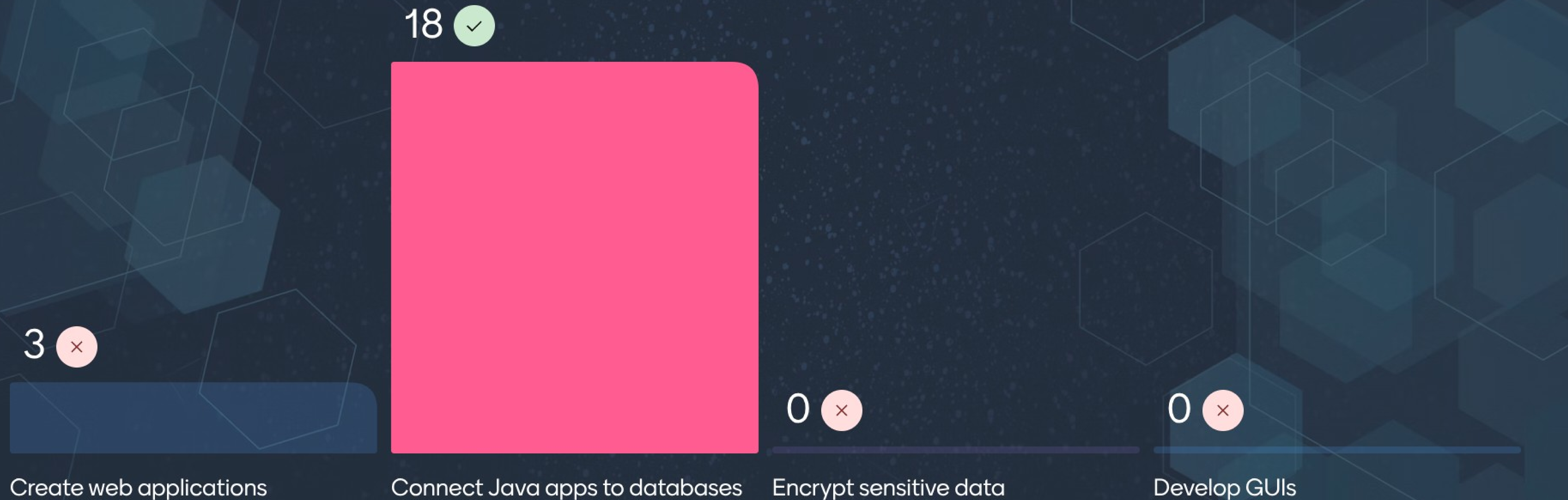


Quiz time

QUIZ



What is the main use of JDBC?



What is the first step in a JDBC application?

0 ✕

Open a file stream

1 ✕

Close database connection

22 ✓

Load the JDBC driver

0 ✕

Write a query

What does the Connection interface represent?

1 ✕

Executing SQL
commands

0 ✕

Updating driver libraries

22 ✓

A session with a
database

1 ✕

Loading a database
schema

How do you create a JDBC connection?

0 ✗

By using SQL command

0 ✗

By parsing a URL manually

0 ✗

Assigning a session ID

9 ✓

Using
`DriverManager.getConnection()`

What does a Statement object do in JDBC?

21 ✓



Executes SQL queries

1 ✗

Handles exceptions

1 ✗

Closes a connection

0 ✗

Modifies the database schema

Which method is used to execute an SQL query?

24 ✓



0 ✗

executeCommand()

0 ✗

executeScript()

1 ✗

runStatement()

What does the ResultSet interface represent?

0 ✗

Database connections

1 ✗

SQL commands

0 ✗

File pointers

22 ✓

Data returned by a query

What does PreparedStatement prevent?

23 ✓



1 ✗

Multi-threaded issues

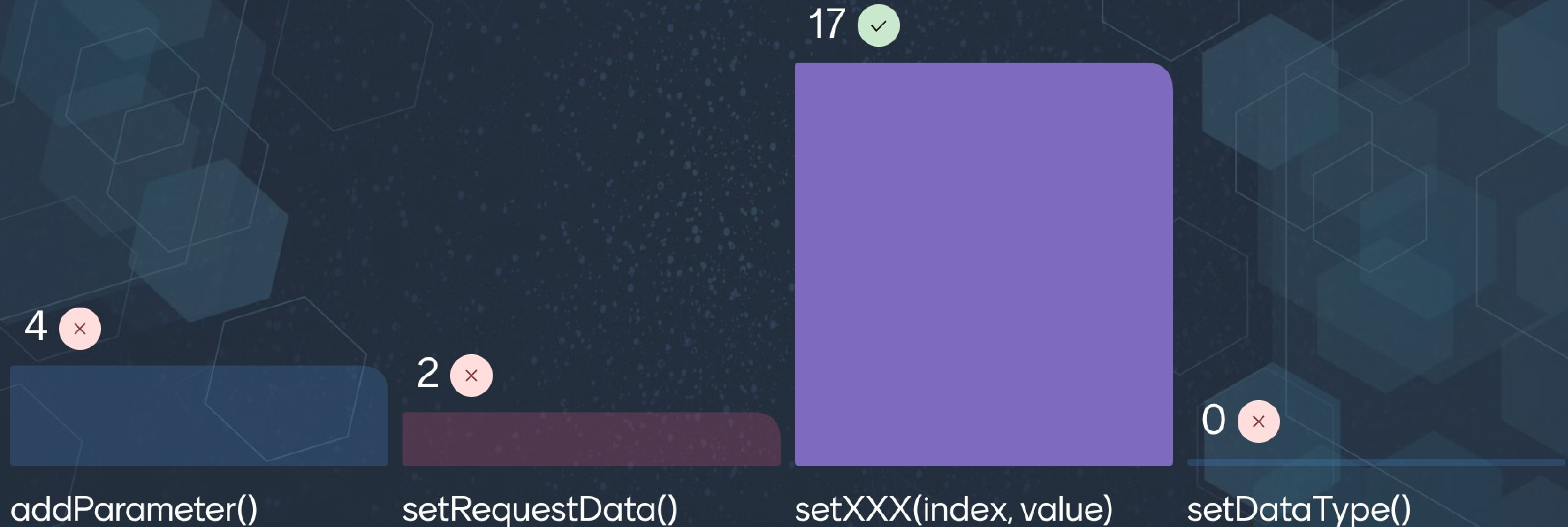
0 ✗

Virtual machine crashes

0 ✗

Semaphore blocking

What method sets parameters for PreparedStatement?



What does `statement.close()` do?

21 



Closes the Statement object

0 

Refreshes the connection

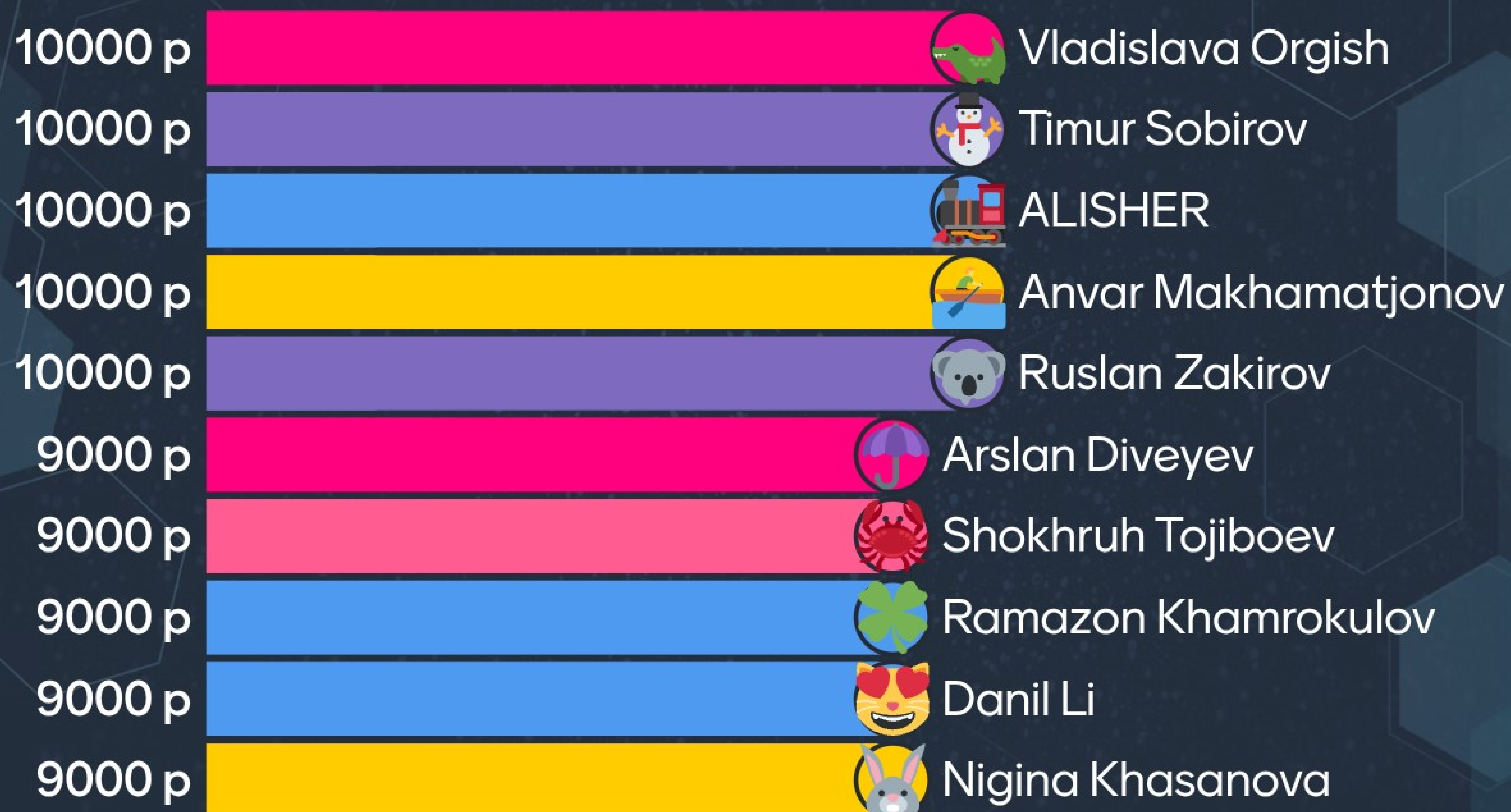
0 

Creates a new object mapping

0 

Re-executes the query

Quiz leaderboard



Q&A part

0 questions
0 upvotes

